Homework 4

Total Marks: 70 **Deadline:** 28 April 2026

General Instructions: You may partner with up to three other students (**a group of max 4**) to submit a single write-up in a single group. We encourage discussion with other students, even if they are not in your group. Each student must build a good understanding of all the questions even if they discuss and collaborate with others. Merely dividing the problems among group members and putting those together before submission will severely impact your learning, and we **strongly advise against it**. Furthermore, while it is okay to use online resources, including **Generative AI** such as **ChatGPT**, for learning purposes, **blatant copying** or **mindlessly querying** the Generative AI is **forbidden**. If there is any confusion or questions, post those on **WhatsApp** group or see your TA or Instructor.

Submission Instructions: Typed or hand-written solutions (only in PDF format) can be submitted on LMS before the deadline. Email submissions WILL NOT be accepted. There will be NO late days or extensions. *Only one submission per group is needed; the first page must clearly mark the names and students IDs of all group members.*

Question 1 SegWit Fundamentals: Transaction Structure, Malleability, and Witness Programs (10)

Segregated Witness (SegWit) changed the way digital signatures and script arguments are stored inside Bitcoin transactions.

a) Explain what data SegWit removes from the traditional scriptSig and where this data goes in a SegWit transaction. Why does this separation reduce transaction malleability? (2)

b) Consider a SegWit pay-to-witness-public-key-hash (P2WPKH) output with witness program:

0x00 || HASH160(pubkey).

Construct the spending transaction fields:

- The scriptSig of the input,
- The witness stack items,
- A short explanation of how the P2WPKH script is evaluated. (4)

c) SegWit introduces the concept of "witness weight" and defines the block weight metric:

weight = (non-witness bytes × 4) + (witness bytes × 1).

Explain why witness data is discounted and give one practical effect on transaction fee incentives. (2)

d) Describe how SegWit enables future extensibility of script versions without requiring a hard fork. Provide one example of how witness versioning allows new script types to be introduced in a backward-compatible way. (2)

Solution:

(a) Separation of signature data and malleability reduction. In legacy transactions, the scriptSig contains signatures and public keys. SegWit removes these variable, user-generated elements from the input and moves them into a separate *witness* section. Since the transaction ID is computed from the non-witness portion only, changing signatures or script arguments no longer changes the transaction ID. This prevents third parties from modifying signatures and creating mutated transaction IDs, solving the classical form of transaction malleability.

(b) Spending a P2WPKH output.

For a P2WPKH witness program 0x00 <20-byte-hash>:

- **scriptSig**: The scriptSig is *empty* (just a push of 0 bytes), typically encoded as:

$$\text{scriptSig} = 0x00.$$

- **Witness stack**: The witness contains:

$$\langle \text{signature} \rangle , \langle \text{public key} \rangle .$$

- **Evaluation:** Internally, the P2WPKH script is interpreted as an implicit P2PKH script:

  OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG.

  During validation, the witness items are used as if they were the scriptSig arguments for this script, completing the signature check.

(c) Weight discount and fee implications. Witness data is discounted because it does not contribute to long-term UTXO set growth and does not affect future chain validation costs. By giving witness bytes a weight of 1 instead of 4, SegWit incentivizes:

- more efficient use of signatures,
- adoption of SegWit output types,
- smaller effective fees for transactions heavy in witness data.

(d) Extensibility through witness versions. SegWit introduces a versioned witness program: <version> <program>. Nodes that do not understand a future version simply treat it as a “pay to anyone who knows the rules of this version,” ensuring old nodes remain

valid. For example, version 0 corresponds to P2WPKH and P2WSH. A future version can introduce new semantics or opcodes, and older nodes will still relay and accept the transaction because the witness program format itself is backward compatible.

Question 2 Bitcoin Scaling for Payments and Privacy Considerations (12)

Bitcoin's base layer has limited throughput but supports several techniques to scale payments and improve user privacy.

a) Describe why Bitcoin's base layer cannot scale to global, high-frequency payments. Include discussion of block size, block interval, and validation constraints. (2)

b) Explain how the Lightning Network improves payment scalability. In your answer, describe:

- how off-chain payments reduce on-chain load,
- how multi-hop routed payments work, and
- the role of HTLCs in atomic forwarding. (3)

c) Identify two privacy weaknesses of base-layer Bitcoin transactions related to:

a. input/output linkability,

b. address reuse. (2)

d) Give two techniques—wallet-level or protocol-level—that improve privacy for everyday Bitcoin payments. Explain briefly how each helps. (2)

e) Compare the privacy characteristics of on-chain payments versus off-chain Lightning payments. In particular, who learns the payment details in each case, and what information is hidden from observers? (3)

Solution:

(a) Base-layer scaling limits. Bitcoin has a fixed block size and an average 10-minute block interval, which caps the number of transactions per second. Validators must verify every transaction and update the global UTXO set, so increasing block size or frequency too much would raise hardware and bandwidth requirements and harm decentralization. Thus the base layer cannot handle global, high-frequency payments by itself.

(b) Lightning for scalable payments. The Lightning Network moves most activity off-chain:

- Two users open a channel with a single on-chain funding transaction. They exchange many payments privately without touching the blockchain.
- Multi-hop routing allows a payment to travel across several channels without requiring direct connections between the sender and receiver.

- HTLCs ensure atomicity: either every hop receives the correct payment or none of them do. This enables trust-minimized routing across many participants.

Only channel openings and closings require on-chain transactions, greatly increasing throughput.

(c) Base-layer privacy weaknesses. *Input/output linkability:* Inputs and outputs within a transaction can reveal common ownership, especially when many inputs are merged. *Address reuse:* Reusing an address makes it easy to link multiple payments and identify user balances.

(d) Privacy-improving techniques. Two examples:

- Using fresh addresses for each payment: prevents linking multiple incoming payments to the same identity.
- Payment batching or collaborative transactions (e.g., PayJoin): obscures which inputs and outputs belong to which participant, making chain analysis more difficult.

Other possible techniques include coin selection strategies, using Tor for network privacy, or off-chain payments.

(e) On-chain vs. off-chain privacy. On-chain payments are public: all inputs, outputs, and amounts are visible to every observer. Anyone can analyze transaction flows. Lightning hides most details:

- Only the channel funding and closing transactions appear on-chain.
- Intermediate Lightning hops learn that a payment is passing through them but not the sender's identity, receiver's identity, or final amount beyond the forwarded value.
- External observers cannot see that most Lightning payments occurred at all.

Thus Lightning offers significantly better privacy compared to on-chain payments.

Question 3 MAST (Merkelized Abstract Syntax Trees) and Script Commitments (10)

Taproot uses MAST to commit to multiple possible spending scripts while revealing only the branch that is actually executed. This greatly improves privacy and reduces on-chain data.

a) Explain the key idea behind MAST. In your answer, describe how Taproot commits to a set of possible scripts, how only the executed script branch is revealed, and why this provides better privacy than traditional P2WSH script commitments. (3)

b) Consider a Taproot output with three possible spending conditions represented by the following witness scripts:

$$S_1 = \text{<pkA> OP\_CHECKSIG},$$

$$S_2 = \text{OP\_IF <pkB> OP\_CHECKSIG OP\_ELSE <pkC> OP\_CHECKSIG OP\_ENDIF},$$

S_3 = <pkD> <pkE> 2 OP_CHECKMULTISIG.

Construct the MAST (Merkle tree) over the *script hashes* of these three scripts. (2)

- Compute the leaf hashes $h_1 = H(S_1)$, $h_2 = H(S_2)$, $h_3 = H(S_3)$.
- Draw the Merkle tree assuming a standard taptree-style Merkle construction (no leaf duplication).
- Compute the internal node hashes and the final Merkle root M.

c) Suppose the spender uses only script S_2. Describe exactly what information must be included in the witness for a script-path Taproot spend, and show the Merkle proof path the spender must provide using the tree you constructed in part (b). (3)

d) Explain why, even though three scripts exist, a chain observer learns nothing about S_1 or S_3 when S_2 is used for the spend. (2)

Solution:

(a) Idea behind MAST. A Taproot output can commit to many alternative witness scripts via a Merkle tree ("taptree"); at spend time, the spender reveals *only* the executed leaf script and a short Merkle proof to the committed root. This hides the existence and content of unused branches, improving privacy and reducing data compared to P2WSH (which reveals the entire witness script).

(b) Building the taptree over script hashes (no duplication). Let

$$h_1 = H_{\text{leaf}}(S_1), \quad h_2 = H_{\text{leaf}}(S_2), \quad h_3 = H_{\text{leaf}}(S_3).$$

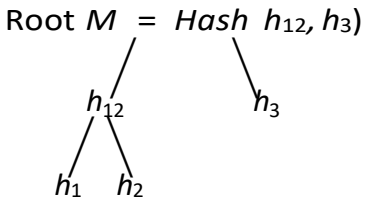
Combine leaves using Taproot's branch rule (lexicographic order):

$$h_{12} = Hash\ h_1, h_2).$$

Then the Merkle root is

$$M = Hash\ h_{12}, h_3).$$

ASCII sketch (no duplication):



(c) Spending with S_2 (script path). A Taproot *script-path* spend must reveal: (i) the *leaf arguments* needed to satisfy S_2, (ii) the *leaf script* S_2 itself, and (iii) a *control block* proving that S_2 is a valid leaf of the committed taptree.

The witness therefore has the form:

$$\langle \text{ args for } S_2 \rangle \ \langle S_2 \rangle \ \langle \text{ control block with Merkle path} \rangle .$$

For our three-leaf tree,

$$h_1 = H_{\text{leaf}}(S_1), \qquad h_2 = H_{\text{leaf}}(S_2), \qquad h_3 = H_{\text{leaf}}(S_3),$$

and

$$h_{12} = H_{\text{branch}}(h_1, h_2), \qquad M = H_{\text{branch}}(h_{12}, h_3).$$

Thus, the Merkle path for S_2 consists of its two siblings:

$$h_2 \xrightarrow{\text{combine with } h_1} h_{12} = H_{\text{branch}}(h_1, h_2), \qquad h_{12} \xrightarrow{\text{combine with } h_3} M = H_{\text{branch}}(h_{12}, h_3).$$

The control block encodes these siblings in order (along with the tapleaf version and internal key), allowing validators to recompute M and verify it matches the output's Taproot commitment.

(d) What the chain observer learns. Only S_2 and its short proof are revealed; no information about S_1 or S_3 (their existence, structure, or keys) is leaked. Non-executed branches remain hidden by construction—this is the central privacy benefit of MASTs.

Question 4 HTLC Scripts and Time-Locked Spending Under Taproot (12)

Hashed Time-Locked Contracts (HTLCs) allow conditional payments based on the revelation of a preimage or on a timeout refund. Taproot enables implementing HTLCs either through script-path spending (with a taptree leaf) or through cooperative key-path spending when both parties agree.

a) Explain the purpose of an HTLC in a multi-hop payment setting (e.g., Lightning). In your answer, describe the roles of the hashlock and timelock, and why the timelock must be *relative* or *absolute* depending on the protocol design. (2)

b) Consider a Taproot output between Alice and Bob with internal key P and a taptree consisting of a single script leaf representing the HTLC. Let the payment hash be $h = H(x)$ and let the timeout be enforced by a timelock T . Construct a Taproot-compatible script-path HTLC leaf script that allows:

- Alice to spend the output *if she learns the preimage x before time T*, and
- Bob to refund the output *after time T*.

You may assume standard opcodes such as:
OP_CHECKSIG, OP_HASH160, OP_EQUAL, OP_CHECKLOCKTIMEVERIFY, and OP_DROP. (3)

c) Draw the Taproot taptree for this output (one script leaf), label the leaf hash, and show how it combines with the internal key P to form the tweaked Taproot output key Q. (2)

d) Suppose the HTLC output is spent cooperatively via the *key-path* with a single Schnorr signature. What information does a blockchain observer learn about:

a. whether this was an HTLC at all,

b. whether a timelock existed,

c. whether a hash preimage was involved?

Explain why Taproot provides this level of privacy. (2)

e) Suppose Alice spends via the script-path before time T using the preimage x. Write the full witness stack (in logical order) required for this Taproot script-path spend, including the Merkle proof/control block. (3)

Solution:

(a) HTLC purpose; hashlock vs timelock. An HTLC enables conditional payment across multiple hops: the receiver discloses a preimage x such that $H(x) = h$ (the *hashlock*), and intermediate hops are paid only if they learn x. The *timelock* guarantees refund if x is not revealed in time. Protocols use an *absolute* timelock (CLTV) to specify a block height or time, or a *relative* timelock (CSV) to require waiting a duration after a prior confirmation.

(b) Taproot-compatible HTLC leaf (script path). Let $h = H(x)$ and timeout T. A standard (P2WSH-style) HTLC leaf adapted for a Taproot taptree is:

S_{HTLC} :
OP_IF
OP_SHA256 ⟨ h⟩ OP_EQUALVERIFY
⟨ pkA⟩ OP_CHECKSIG
OP_ELSE
⟨ T⟩ OP_CHECKLOCKTIMEVERIFY OP_DROP
⟨ pkB⟩ OP_CHECKSIG
OP_ENDIF

(c) Taptree and Taproot output key. With a single leaf, the tapleaf hash $h_{\text{leaf}} = H(\text{serialize}(S_{\text{HTLC}}))$ is the Merkle root M. The Taproot output key is the tweak of the internal key P:

$$Q = P + H(P \,\|\, M) \cdot G,$$

and the output is encoded as the SegWit v1 witness program OP_1 ⟨ 32-byte data⟩ (the 32 B encoding of Q). Only the used branch is later revealed.

(d) Privacy with key-path cooperation. If Alice and Bob cooperate and spend via the *key path* (Schnorr aggregate signature), the chain observer sees only a single signature for Q and cannot tell (i) that this was an HTLC, (ii) that a timelock existed, or (iii) that a preimage might have been involved. Key and signature aggregation make multisig or script use indistinguishable from single-sig, improving privacy.

(e) Script-path spend (preimage path) witness. For spending *before* T using x, the witness stack (logical order) is:

⟨ sigA⟩ ⟨ x⟩ ⟨ OP TRUE⟩ ⟨ S_{HTLC}⟩ ⟨ control block (with empty/one-element Merkle path)⟩ .

Explanation: (1) The selector OP_TRUE chooses the hashlock branch (OP_IF).
(2) The script verifies OP_SHA256 ⟨ x⟩ equals ⟨ h⟩ and then checks sigA against pkA.
(3) The control block proves that S_{HTLC} is a valid leaf under the committed taptree for Q
Refund path (after T): the witness would be

⟨ sigB⟩ ⟨ (empty)⟩ ⟨ OP FALSE⟩ ⟨ S_{HTLC}⟩ ⟨ control block⟩ , so the OP_ELSE branch executes, enforcing CLTV and verifying sigB.

Question 5 Lightning Revocation Mechanisms and Punishment for Old-State Broadcasts (12)

Two-party payment channels require both parties to update their channel balances off-chain while preventing either participant from broadcasting an outdated commitment transaction. The Lightning Network uses a revocation-based punishment mechanism to enforce correct behavior.

a) Explain why outdated channel states (old commitment transactions) are dangerous. Describe what would happen if a party could safely publish an earlier state without penalty. (2)

b) In the simplified Lightning model used in class, each commitment transaction contains a *revocation key* whose private secret is revealed only when that state becomes old. Explain how a new state revokes the prior one. In your explanation, include:

- the role of the revocation secret for each state,
- how the counterparty uses it to gain control of the revocation key,
- and why revealing this secret makes an older state unsafe to publish. (3)

c) Suppose Alice and Bob open a channel. When they sign state $i + 1$, Alice reveals the revocation secret for state i, and Bob stores it. If Alice later broadcasts state i, which output can Bob take, and why? Describe how the timelock on that output enables Bob to punish Alice. (3)

d) Construct a simplified script (in Bitcoin Script) for a *revocable output* that pays either:

- the original owner after a timelock expires, or
- the counterparty immediately if they know the revocation secret. (2)

e) Describe what Bob includes in his *punishment (justice) transaction* when Alice broadcasts an old state, and explain why this mechanism creates strong economic incentives to only broadcast the latest state. (2)

Solution:

(a) Why outdated states are dangerous. A channel state represents how the total balance is divided at a particular moment. If a party could safely publish an old state, they

could revert the balance to a version where they owned more funds. This would allow them to steal money by rolling the channel back to a previous (favorable) state, making Lightning channels insecure.

(b) Revoking the prior state using revocation secrets. In the simplified model:

- Each commitment transaction includes a revocation *public key*.
- Each state has a corresponding revocation *secret* that unlocks this public key.

When state $i + 1$ is created, the parties exchange signatures for that new state. *Only after this is done*, the revocation secret for state i is revealed.

Once Bob learns the revocation secret for state i, he can spend the revocation branch of Alice's output in state i. Thus, if Alice publishes state i later, Bob can immediately take the funds from Alice's "to-self" output. Revealing the revocation secret makes all earlier states unsafe to broadcast.

(c) Bob's punishment using the delayed output. Each commitment transaction contains:

- a *delayed* output for the broadcaster (e.g., Alice), and
- an *immediate* revocation output for the counterparty (Bob).

If Alice broadcasts outdated state i, Bob already knows the revocation secret for that state, because Alice revealed it when state $i + 1$ was agreed upon. Using this secret, Bob can spend the revocation path *immediately*. Alice's own output is timelocked, so she must wait (e.g., via CHECKSEQUENCEVERIFY), giving Bob time to take the entire output. Thus Bob claims Alice's revocable output as punishment.

(d) Revocable output script. A simplified script implementing this behavior is:

```
OP_IF
    <revocation_pubkey>
OP_ELSE
    <delay>  OP_CHECKSEQUENCEVERIFY  OP_DROP
    <owner_pubkey>
OP_ENDIF
OP_CHECKSIG
```

Interpretation:

- Counterparty spends immediately using the revocation private key (the secret revealed for an old state).
- Original owner can spend only after waiting the relative timelock.

(e) The justice transaction and incentives. When punishing Alice, Bob creates a *justice transaction* that:

- spends the revocable output using the revocation secret,
- provides a signature with the corresponding private key,
- takes the *entire* output immediately.

This creates a strong economic incentive:

$$\text{Broadcasting an old state} \Rightarrow \text{you lose } \textit{all} \text{ your funds in that output.}$$

Thus the rational behavior is to always publish the latest state, ensuring honest channel operation.

Question 6 Lightning Network Fundamentals: Payment Channels and Commitment Transactions (14)

Two parties, Alice and Bob, set up a bidirectional payment channel using a funding transaction and off-chain commitment transactions representing their evolving balances.

a) Describe the structure and purpose of the *funding transaction* in a Lightning channel. Why must it be a 2-of-2 multisig output, and how does this fact enable trust-minimized updates? (3)

b) Define what a *commitment transaction* is. Explain:

- who signs it,
- when it is valid to broadcast,
- why each party holds a slightly different commitment transaction. (3)

c) Explain how off-chain payments change the channel balance without broadcasting transactions. Describe the update process that ensures both parties end up with the latest valid state and that older states cannot be safely broadcast. (3)

d) Consider a channel where Alice initially contributes 1 BTC and Bob contributes 0 BTC. They then perform the following off-chain transfers:

a. Alice pays Bob 0.3 BTC,
b. Bob pays Alice 0.1 BTC,
c. Alice pays Bob 0.25 BTC.

Compute the final off-chain channel balance for each party and explain how this is reflected in the final pair of commitment transactions. (3)

e) Give one reason why commitment transactions must include timelocks, and describe what could go wrong if timelocks were not present. (2)

Solution:

(a) Funding transaction. The funding transaction opens the channel. It locks coins in a 2-of-2 multisignature output requiring signatures from both Alice and Bob. This ensures that neither participant can move the funds unilaterally; all channel updates must be cooperatively signed. The 2-of-2 lock establishes the shared pot from which commitment transactions allocate balances.

(b) Commitment transactions. A commitment transaction is a local, fully signed transaction spending the funding output and distributing the current channel balance to Alice and Bob. Each party holds a different version:

- Alice holds a commitment transaction that pays her to a delayed output and Bob to an immediate output.
- Bob holds a symmetric but inverted version.

Each version is valid for *only one* of the parties to broadcast—this prevents one party from unilaterally biasing the output ordering or script structure. A commitment transaction is broadcast only if the channel is closing or if one party misbehaves.

(c) Updating channel balances. To update the channel:

a. Both parties create a new pair of commitment transactions reflecting the new balances.

b. They exchange signatures so the new state becomes valid for broadcast.

c. Only after signatures are exchanged, each reveals the *revocation secret* for the *previous* state.

Revealing the old per-state secret makes the previous commitment transaction punishable if broadcast, so neither side can revert to an older balance. Thus, balances change entirely off-chain without touching the blockchain.

(d) Final channel balance. Starting from:

$$\text{Alice: } 1.00 \text{ BTC}, \qquad \text{Bob: } 0.00 \text{ BTC}.$$

1. Alice pays Bob 0.3 BTC:
$$A = 0.70, \quad B = 0.30.$$
2. Bob pays Alice 0.1 BTC:
$$A = 0.80, \quad B = 0.20.$$
3. Alice pays Bob 0.25 BTC:
$$A = 0.55, \quad B = 0.45.$$

Final balances:

$$\boxed{A = 0.55 \text{ BTC}, \qquad B = 0.45 \text{ BTC}.}$$

In the final commitment transactions:

- Alice's version gives Alice 0.55 BTC (delayed) and Bob 0.45 BTC (immediate).
- Bob's version gives Bob 0.45 BTC (delayed) and Alice 0.55 BTC (immediate).

(e) Role of timelocks. Timelocks ensure that the broadcaster of the commitment transaction *cannot* immediately take their own output; they must wait a relative delay. This delay gives the counterparty time to punish them if the broadcaster attempted to use an old revoked state. Without timelocks, publishing an outdated commitment transaction would be safe, completely breaking Lightning's security.